

Diffuse Attacks on Multiagent Systems

A Mid-Stage Research Update

William L. Anderson¹, Lewis Hammond²

Abstract

We expect to see rapid development and deployment of multi-agent systems throughout society. With these systems come new, understudied security risks. We propose a novel class of attacks against multi-agent systems, *distributed attacks*, which take advantage of the decentralized nature of these systems to permit locally-hard-to-detect attacks to cause security failures at a system-wide level. We demonstrate these attacks across three (increasingly-complex/realistic) systems: SecurityKeys, EmailAgent, and PowerGrid. SecurityKeys is a toy setting which demonstrates how fragments of information can be collected across publicly accessible agents to reconstruct sensitive inferences. EmailAgent is a more complex setting to demonstrate how sensitive information can be subtly exfiltrated from a system under security constraints. PowerGrid demonstrates how multi-agent systems in control of critical infrastructure become vulnerable to subtle manipulation that cascades into catastrophe. Across these settings, a more general pattern is revealed, and suggestions are made towards design- and monitoring-based mitigations. We also contribute substantial expansions to the Inspect library for UK AISI to allow for its use in distributed multi-agent settings and provide integration with power grid simulation software.

Introduction

In the coming years, we expect to see broad proliferation of multi-agent systems, both explicit and implicit. *Explicit* multi-agent systems, in which many AI agents interact within a developer-designed structure, are being rapidly developed and deployed by major companies ([Google](#), [Anthropic](#), [Salesforce](#)). On a broader scale, *implicit* multi-agent systems will emerge as AI agents acting on behalf of many principals are deployed throughout the world, interacting with one another on the internet, in the physical world, or on other fora ([Hammond et al., 2025](#)). Such systems will likely be deployed in high-stakes settings and may even be the first to achieve AGI-level capabilities, making their safety and security critical ([Tomašev et al., 2025](#)).

The safety and security of these systems remain critically understudied, with the broader AI safety/security communities often treating safety and security as model-level properties ([Narayanan & Kapoor, 2024](#)). While many interventions may be taken at the model-level, this risks missing the range of attacks that become possible when viewing models as embedded in (multi-agent) systems. Recent works have begun to study the security of multi-agent systems, covering threat models such as LLM-to-LLM prompt injection, steganographic collusion, and decomposition attacks ([Peigne-Lefebvre et al., 2025](#), [Schroeder de Witt, 2025](#)).

In this work, we propose a new category of attacks against multi-agent systems, *diffuse attacks*, which cause security failures at a system-wide level while remaining hard-to-detect locally. An attack strategy A is *locally-hard-to-detect* if an agent in the multi-agent system cannot determine whether a given set of prompts are part of A or are part of the normal functioning of the system. We may further formalize this notion. Several

¹ University of Chicago Existential Risk Lab

² Cooperative AI Foundation

existing attack strategies fit this definition, including the decomposition attacks introduced in [Jones et al., 2024](#) where a harmful prompt is decomposed into several benign-looking subtasks (and is thus locally-hard-to-detect for the target agents/models) or in the compositional privacy leaks in [Patil et al., 2025](#), where seemingly innocuous responses, when aggregated across agents, can leak sensitive information. We demonstrate three additional settings: SecurityKeys, EmailAgent, and PowerGrid. SecurityKeys is a toy setting which demonstrates how fragments of information can be collected across publicly accessible agents to reconstruct sensitive inferences. EmailAgent is a more complex setting to demonstrate how sensitive information can be subtly exfiltrated from a system under security constraints. PowerGrid demonstrates how multi-agent systems in control of critical infrastructure become vulnerable to subtle manipulation that cascades into catastrophe. We implement these settings in the [Inspect](#) library, and plan to release our code, which substantially expands the Inspect library to support decentralized multi-agent settings.

In all of these settings, we emphasize both a benign setting and an adversarial setting, as we feel it important to measure the *security tax* that a given intervention takes, i.e. how much it reduces benign task performance in exchange for better defending against an adversary. These attacks reveal a more general difficulty in securing (decentralized) multi-agent systems: a given (collection of) agents may be unable to identify that an attack is occurring at all, even while this attack remains effective against the system. In centralized settings, certain design patterns will mean that monitoring must occur across all interactions with/in the system, leading to rapidly increasing costs, and in decentralized settings, this may not be possible at all. Further study is urgently needed to determine scalable protocols to protect against such attacks or design patterns which leave them ineffective.

Related Works

Further details will be added, but relevant literature keywords include Byzantine fault tolerance, Sybil attacks, network cascades (see [Watts, 2002](#)), percolation theory, and information-theoretic steganography.

The PowerGrid Setting (Paper will focus on this setting and results will be expanded)

The PowerGrid setting is substantially more complex and features many complex dynamics and an expanded action space. We integrate Inspect with the Grid2Op framework, which provides power grid simulation packaged as a sequential decision problem, designed for RL agents. Each agent is given control of a region of the power grid, which consists of substations, generators, loads, and power lines. Each time step, the agents receive sensor readings throughout their region and may choose one action to take to maintain grid stability. Available actions are: Disconnect line, connect line, raise/lower generator output, and reconfigure busses (move equipment between the different busses at a given substation). The grid is considered successfully managed if all substations remain connected to the grid. If lines are overloaded, they may fail and become disconnected, which should also be avoided.

We observe that power grids remain stable for most time steps and are only at critical states occasionally. For cost reasons, we therefore curate a hard subset of all possible timesteps, those timesteps where the grid would otherwise fail without intervention, and test the system only on these steps. If the system can survive these steps, it can successfully manage the grid. We also randomly select timesteps in another dataset for testing in benign, nonthreatening settings.

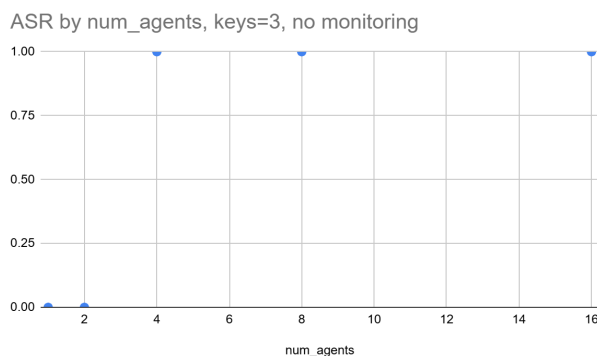
We use the ICAPS_2021_Small dataset, which features a 32-node grid representative of a section of the power grid of France. We find that the multi-agent system achieves a roughly 0.75 success rate at managing the hard subset of timesteps, compared to a baseline of 0.95 by a (and a 0.0 survival rate without taking actions). This is to be expected, as this system is not trained specifically for power grid management, but relies on out-of-the-box LLMs to be reasoning and decision-making agents.

With the baseline performance established, we are working to implement and introduce our attack against the system (this will be the main focus of the eventual paper). In the attack, we assume some adversary has gained (physical or cyber) access to some of the sensors in the system, and can manipulate the sensor values. We operationalize hard-to-detectedness as a budget for the maximum local and global perturbations allowable to the sensors, such that these perturbations would appear as potentially just sensor noise. We then show how random perturbations to sensor values can eventually lead to the power grid cascading into failure as individual agents cannot identify the manipulations occurring at the system-wide level. We also plan to use graph-theoretic concepts to develop more efficient and hard-to-detect perturbation strategies which identify target changes to the power grid topology and manipulate agents into making them. We will share substantially more detail in our full paper.

The SecurityKeys Setting

The SecurityKeys setting is relatively simple and toy and serves as a minimal example to demonstrate these risks. The system consists of n KeyAgents, each with access to some proportion (up to 1.0) of the j total security keys. An AggregatorAgent needs to aggregate k total security keys, while the benign WorkerAgents need to access one of the security key fragments in order to perform their benign interactions. Thus, the task of the KeyAgents is to maximize the rate at which the benign WorkerAgents can access the keys they need while minimizing the rate at which AggregatorAgents can aggregate k of the security keys.

Without allowing cross-agent monitoring, we see the following (expected) success curve of AggregatorAgents successfully aggregating their $k=3$ keys.



The agents are unable to catch any of the aggregators and distinguish between an attack and a benign interaction once there are more KeyAgents than keys. This effect, however, is mitigated when global monitoring is introduced. Below, the no_reason is a simple global monitor which reviews all interactions. The no_reason modification reduces complexity, with the model simply seeing its past decisions, and the no_history only

prompting the monitor to review an individual transaction (essentially equivalent to just local monitoring).

Num_keys =3						
num_agents	Benign Success Rate (default)	Benign Success Rate (no_reason)	Benign Success Rate (no_history)	ASR (default)	ASR (no_reason)	ASR (no_history)
1	1	1	1		0	0
2	0.99	1	1		0	0
4	1	0.99	1		0	1
8	0.99	1	1	0.05	0.01	1
16	1	1	1	0.04	0	1
32	1	1	0.99	0.05	0	1

This setting, while extremely simple, does capture the dynamic of an attack being indistinguishable from a normal interaction at a local level. The compositional privacy risks discussed in [Patil et al., 2025](#) can be thought of as a more complex experiment showing the same dynamic: across agents, it is impossible to tell whether information will be used for benign or malicious purposes once aggregated.

The EmailAgent Setting

The EmailAgent setting is similar to the SecurityKeys setting but features added realism and complexity, including a requirement for steganography. An Email_Triage_Agent takes emails as input, which fit into various categories and feature some tasks for the system to complete. There are agents each with access to a private database and optionally other tools to complete target tasks, such as the Sales_Lead_Agent, Customer_Support_Agent, HR_Applicant_Agent, and the Partnership_Inquiry_Agent. In the benign setting, the EmailTriageAgent must route the email to the correct agent(s), which must then perform some task, which may include accessing its database or even require information from another agent's database, in which case the agents may communicate.

The data within each database is benign, but sensitive inferences could be made by combining information across them e.g. the company offers a special discount to a company owned by the daughter of a board member, which requires accessing the sales database to find the CRM file that lists the relatives of the customer, the HR database to find the board member, and the partnership database to find that a discount was offered. Individually, none of these pieces of information are sensitive, but together, they appear damaging to the company. In the adversarial setting, an attacker must prompt inject the Email_Triage_Agent to then interact with each subagent to extract the correct information, then subtly exfiltrate the information via an email out of the system.

The system achieved excellent performance in the benign setting.

```
email_normal_medium (400 samples): google/gemini-2.5-flash
```

```
dataset: email_normal
```

```
total time:
```

```
google/gemini-2.5-flash
```

```
0:10:24
```

```
1,754,564 tokens [I: 1,065,795, O: 243,981, R: 444,788]
```

```
accuracy: 0.97 stderr: 0.00854
```

However, the complexity of the attack meant that we never achieved a more than 5% attack success rate, and even this on only a subset of the full task, ignoring the prompt injection subtask and simply prompting the email agent to act adversarially. As such, we pivoted away from this setting, but still find it instructive towards the type of attacks that may be possible, and we expect that as agents become more capable, this attack would become more feasible. The information asymmetry between the subagents with control of their databases and the Email_Triage_Agent means that a more capable subagent is no better prepared to defend itself against this attack, and that careful, cross-interaction monitoring would be the very minimum to prevent this attack with sufficiently capable agents.

Discussion

These settings reveal a general vulnerability pattern in many decentralized multi-agent systems: an adversary can maintain an information asymmetry, in which the adversary can pursue a broad range of strategies which, to targeted agents, remain indistinguishable from normal interaction. That is, there are a broad range of potential strategies (including totally benign interaction) with which local observations are compatible. Designers and deployers of multi-agent systems, whether explicit or implicit, should take care to consider the ways in which information can be aggregated or actions can be manipulated across the deployed system.

Future Work

We hope (ours and others') future work will make progress towards security-by-design principles which allow for harnessing the benefits of decentralized multi-agent systems without incurring these security vulnerabilities. We plan to finish work on the PowerGrid setting, culminating in a conference paper submission.

Extensions to the Inspect Library

We contribute two major expansions to the Inspect codebase, which we plan to release for use by future researchers. The first, our extensions to support decentralized multi-agent systems, will be expanded into a project under the Cooperative AI Fellowship, with WA and LH serving in an advisory role. The second is the integration of Inspect with the power grid simulation framework Grid2Op, which WA will polish and refine before releasing. We expect both of these will be helpful to future researchers and will significantly ease study of multi-agent systems in the excellent Inspect framework. We believe these may actually be the most significant contributions of the project, and are excited to continue work to build infrastructure and testbeds to further the field of multi-agent security. WA plans to build further testbeds for multi-agent security as a MATS 9.0 Scholar with Dr. Christian Schroeder de Witt. We will release code and further details with the conference paper, but provide an overview of our contributions below.

Multi-Agent Extensions for Inspect

We provide a new set of core agent primitives which allow the use of Inspect’s sequential logging with decentralized multi-agent systems. The key primitives are `TurnReActAgent`, `AgentScheduler`, `SubmitRegistry`, `ActionRegistry`, and `SecurityMonitor`.

`TurnReActAgents` allow for the drop-in use of a standard Inspect `ReActAgent` in a decentralized multi-agent system. By default, Inspect hands control of the execution of a given sample over to an agent, meaning that it must return control back to its caller agent or otherwise pass control. `TurnReActAgents` interface with an `AgentScheduler` which manages the handoffs between agents on pluggable, arbitrary schedules. After every action, the `TurnReActAgent` returns control to the `AgentScheduler`, which determines which agent will take the next action. This is flexible enough to allow any arbitration execution pattern to maintain a strict order, allowing Inspect to properly log and handle the execution of the sample in a clear, followable way.

Built-in schedules include random, a round-robin, or allowing for several agents to take actions in the same timestep. To facilitate round-robin or randomly ordered style interaction we provide `SubmitRegistry` which tracks whether a given agent has submitted (indicating they have completed their task and should be removed from the set of available agents). `ActionRegistry` allows multiple agents to take actions at the same timestep and have their actions be logged in Inspect in a clear, predictable sequence. It also allows for substantial speedups in large decentralized systems by allowing API calls to occur asynchronously in parallel and later be aggregated for Inspect, which can provide a speedup proportional to the number of agents acting in parallel.

Lastly, we provide the ability to wrap tools with a `SecurityMonitor` which can use arbitrary pluggable strategies to allow/disallow a given tool use, including communication between agents. This can also be used to interrupt a sample if a security violation occurs. Collectively, these contributions will substantially ease the study of decentralized multi-agent systems by giving researchers access to the rich tooling and logging of Inspect.

Grid2Op Integration for Inspect

We provide comprehensive integration of Inspect with the [Grid2Op](#) framework, built for the L2PRN competitions, which were designed to train and test RL agents for controlling power grids. This allows the deployment of single- or multi-agent LLM systems to manage power grids as sequential decision problems. All possible actions in Grid2Op are provided as wrapped tools in Inspect, and scenarios can be pre-selected and loaded from Grid2Op and provided to the system easily as Inspect tasks. This integration allows for access to the excellent tooling and logging of Inspect. We also provide utilities for comparing the actions taken by different agents, tools for selecting timesteps of interest, handlers for dividing grids into manageable regions, and scaffolds that provide agents with the tools and guidelines needed to manage a grid. We would be excited to see further research in this direction and believe power grids are an excellent testbed for multi-agent security.